

Studi Kasus A. String Cantik OSN-P

Mari kita bedah aturan string cantik OSN-P dan jawab **Soal A1**, **Soal A2**, dan **Soal A3** satu per satu.

Key Rules (Aturan Utama):

1. **Karakter:** Hanya terdiri dari karakter 'O', 'S', 'N', dan 'P'. Masing-masing huruf harus muncul **minimal 1 kali**.
2. **Posisi 'P':** Tidak boleh ada huruf selain 'P' setelah huruf 'P'. Artinya, begitu karakter 'P' pertama muncul, seluruh karakter setelahnya harus berupa 'P' (semua huruf 'O', 'S', dan 'N' wajib berada di sebelah kiri huruf 'P' pertama).

Soal A1

Pertanyaan: Manakah dari pilihan berikut yang merupakan string cantik OSN-P?

- **A. "OSN" → Salah** (tidak memiliki huruf 'P').
- **B. "SSNOSOPP" → BENAR**
 - Terdiri dari huruf 'O', 'S', 'N', dan 'P' (semuanya muncul ≥ 1 kali).
 - Karakter setelah 'P' pertama hanya ada 'P'.
- **C. "OSNPOP" → Salah** (terdapat huruf 'O' setelah huruf 'P').
- **D. "NOWOSP" → Salah** (mengandung huruf 'W' yang bukan anggota OSNP).
- **E. "OSOPP" → Salah** (tidak memiliki huruf 'N').

Jawaban A1: B**Soal A2**

Pertanyaan: Diberikan $S = \text{"SOPRANOSATPOLPP"}$. Apakah Pak Dengklek dapat membuat string cantik OSN-P yang memiliki **tepat 3 huruf 'P'**?

Mari kita periksa huruf 'P' pada string S :

- Karakter 'P' muncul pada indeks:
 - 'P' ke-1: $\text{"SO[P]RANOSATPOLPP"}$
 - 'P' ke-2: $\text{"SOPRANOSAT[P]OLPP"}$
 - 'P' ke-3: $\text{"SOPRANOSATPOL[P]P"}$
 - 'P' ke-4: $\text{"SOPRANOSATPOLP[P]"}$

Jika kita mengambil tepat 3 huruf 'P', maka 3 huruf 'P' yang kita pilih harus berada di bagian paling akhir dari substring yang kita ambil (karena setelah 'P' pertama tidak boleh ada huruf lain).

- Kita bisa memilih prefix sebelum 'P' ke-2: "SOPRANOSAT" → hapus karakter bukan OSN → kita bisa membentuk "SONOS" (mengandung 'O', 'S', 'N').
- Diikuti 3 huruf 'P' terakhir dari S (di indeks 'P' ke-2, ke-3, ke-4): "PPP" .
- Hasil gabungan: "SONOSPPP" → Ini adalah string cantik OSN-P dengan tepat 3 huruf 'P'!

Jawaban A2: BENAR**Soal A3**

Pertanyaan: Dari $S = \text{"SOPRANOSATPOLPP"}$, apa **string cantik OSN-P terpanjang** yang dapat dibuat? Jika ada lebih dari satu, pilih yang **paling kecil secara leksikografis**.

Langkah 1: Saring Karakter

Hapus semua karakter selain 'O', 'S', 'N', 'P'.

- String awal S : "S O P R A N O S A T P O L P P"
- Setelah menghapus 'R', 'A', 'T', 'L':

$$S' = \text{"S O P N O S P O P P"}$$

Langkah 2: Tentukan Batas Karakter 'P' Pertama

Ingat bahwa setelah huruf 'P' pertama yang kita pertahankan, **seluruh karakter berikutnya yang tersisa HARUS berupa 'P'**.

Mari kita cek opsi titik potong huruf 'P' pertama yang digunakan:

1. **Jika 'P' pertama yang kita pakai adalah 'P' di indeks ke-3 pada S' ("S O P ..."):**
 - Karakter sebelum 'P': "S O" (belum ada 'N').
 - Sisa 'P' setelahnya ada 3 buah.
 - Tidak bisa membentuk string cantik karena tidak ada 'N'.
2. **Jika 'P' pertama yang kita pakai adalah 'P' di indeks ke-7 pada S' ("S O P N O S P ..."):**
 - Substring sebelum 'P' ke-7: "S O N O S" (sudah lengkap ada 'O', 'S', 'N').
 - Karakter 'P' yang tersedia mulai dari indeks ke-7 hingga akhir: ada **3 buah** 'P' (indeks 7, 9, 10 pada S').
 - Maka string yang terbentuk: "SONOSPPP" (Panjang = 8).
3. **Jika 'P' pertama yang kita pakai adalah 'P' di indeks ke-9 pada S' ("S O P N O S P O P P"):**
 - Substring sebelum 'P' ke-9: "S O N O S O" (ada 'O', 'S', 'N').
 - Karakter 'P' yang tersedia dari indeks ke-9 hingga akhir: ada **2 buah** 'P'.
 - Maka string yang terbentuk: "SONOSOPP" (Panjang = 8).

Langkah 3: Cari Panjang Maksimum & Terkecil Secara Leksikografis

Kedua calon string cantik terbaik memiliki **panjang yang sama, yaitu 8 karakter**:

1. "SONOSPPP"
2. "SONOSOPP"

Mari kita bandingkan secara leksikografis (urutan kamus):

- Keduanya diawali dengan "SONOS...".
- Karakter ke-6:
 - "SONOSP..." (karakter ke-6 adalah 'P')
 - "SONOSO..." (karakter ke-6 adalah 'O')
- Karena 'O' muncul sebelum 'P' dalam abjad ($'O' < 'P'$), maka "SONOSOPP" lebih kecil secara leksikografis!

Jawaban A3: SONOSOPP

Soal Pemrograman A

Berikut adalah pembahasan dan solusi C++ yang optimal untuk menyelesaikan problem **Studi Kasus A: String Cantik OSN-P**.

Analisis Masalah

Syarat string cantik OSN-P adalah:

1. Terdiri dari huruf 'O', 'S', 'N', dan 'P'.
2. Tiap huruf minimal muncul 1 kali.
3. Setelah kemunculan huruf 'P' pertama (di dalam string akhir), **hanya boleh ada huruf 'P'**.

Berdasarkan aturan di atas, format dari "string cantik" selalu berupa gabungan beberapa karakter 'O', 'S', dan 'N' di bagian awal, lalu diikuti oleh sekumpulan karakter 'P' hingga akhir.

Untuk memaksimalkan panjang string:

1. Kita tentukan salah satu karakter 'P' pada string asal S untuk menjadi **huruf 'P' pertama** pada string cantik kita.
2. Semua huruf 'O', 'S', dan 'N' yang ada di **sebelah kiri** huruf 'P' tersebut bisa kita ambil seluruhnya (karena hal ini tidak melanggar syarat posisi 'P').
3. Semua huruf 'P' mulai dari titik tersebut **hingga ke ujung akhir string S** juga bisa kita ambil seluruhnya.
4. Agar valid, kita hanya perlu memastikan bahwa pada bagian sebelah kiri huruf 'P' pilihan kita, sudah terdapat setidaknya satu huruf 'O', satu huruf 'S', dan satu huruf 'N'.

Kita dapat menyelesaikan ini dengan iterasi $\mathcal{O}(N)$:

- Hitung terlebih dahulu total kemunculan huruf 'P' pada seluruh string.
- Lakukan iterasi dari kiri ke kanan sambil menghitung kemunculan 'O', 'S', dan 'N'.
- Setiap kali bertemu huruf 'P', anggap huruf tersebut adalah huruf 'P' pertama dari string cantik. Jika syarat jumlah 'O', 'S', dan 'N' minimal 1 terpenuhi, catat panjang totalnya (yakni total 'O', 'S', 'N' sejauh ini ditambah sisa huruf 'P' hingga akhir string).
- Lakukan perbandingan untuk mencari nilai maksimum.

Kode Program C++

C++ versi GCC 17.X atau 20.X

```
#include <iostream>
#include <string>
#include <algorithm>

using namespace std;

int main() {
    // Optimasi I/O untuk mempercepat eksekusi
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    string S;
    if (!(cin >> S)) return 0;

    // Hitung total huruf 'P' di seluruh string S terlebih dahulu
    int cnt_p_remaining = 0;
    for (char c : S) {
        if (c == 'P') {
```

```

        cnt_p_remaining++;
    }
}

int cnt_o = 0;
int cnt_s = 0;
int cnt_n = 0;
int max_len = -1;

// Iterasi string S dari kiri ke kanan
for (char c : S) {
    if (c == 'P') {
        // Jika kita menggunakan 'P' ini sebagai 'P' awalan dari string cantik,
        // pastikan huruf O, S, dan N sudah muncul setidaknya 1 kali.
        if (cnt_o > 0 && cnt_s > 0 && cnt_n > 0) {
            int current_len = cnt_o + cnt_s + cnt_n + cnt_p_remaining;
            if (current_len > max_len) {
                max_len = current_len;
            }
        }
        // Karena kita maju ke indeks selanjutnya,
        // jumlah huruf 'P' yang tersisa (termasuk yang sekarang) akan berkurang
        cnt_p_remaining--;
    }
    else if (c == 'O') {
        cnt_o++;
    }
    else if (c == 'S') {
        cnt_s++;
    }
    else if (c == 'N') {
        cnt_n++;
    }
}

cout << max_len << "\n";

return 0;
}

```

Jika kamu harus melakukan *compile* program ini menggunakan *compiler* lawas seperti **GCC 3.x**, kendala utama yang akan kamu temui adalah *error* pada bagian perulangan `for (char c : S)`.

Fitur tersebut adalah *range-based for loop* yang baru diperkenalkan pada standar **C++11** (tahun 2011), sementara GCC 3.x hanya mendukung standar **C++98/C++03**. Jika di-*compile* di GCC 3.x, compiler akan mengeluarkan *error* seperti `expected ';' before ':'` token.

Solusinya sangat sederhana: kita cukup mengganti perulangan tersebut dengan perulangan `for` konvensional berbasis indeks (menggunakan `S.length()`). Berikut adalah kode yang sudah disesuaikan dan dijamin 100% kompatibel dengan **C++98 (GCC 3.x)**:

```

C++
#include <iostream>
#include <string>

using namespace std;

int main() {
    // Optimasi I/O
    ios_base::sync_with_stdio(false);

```

```

cin.tie(NULL);

string S;
if (!(cin >> S)) return 0;

int cnt_p_remaining = 0;
int len = S.length(); // Simpan panjang string ke variabel

// Iterasi gaya C++98 untuk menghitung total huruf 'P'
for (int i = 0; i < len; ++i) {
    if (S[i] == 'P') {
        cnt_p_remaining++;
    }
}

int cnt_o = 0;
int cnt_s = 0;
int cnt_n = 0;
int max_len = -1;

// Iterasi gaya C++98 dari kiri ke kanan
for (int i = 0; i < len; ++i) {
    char c = S[i]; // Ambil karakter pada indeks ke-i

    if (c == 'P') {
        if (cnt_o > 0 && cnt_s > 0 && cnt_n > 0) {
            int current_len = cnt_o + cnt_s + cnt_n + cnt_p_remaining;
            if (current_len > max_len) {
                max_len = current_len;
            }
        }
        cnt_p_remaining--;
    }
    else if (c == 'O') {
        cnt_o++;
    }
    else if (c == 'S') {
        cnt_s++;
    }
    else if (c == 'N') {
        cnt_n++;
    }
}

cout << max_len << "\n";

return 0;
}

```

Perubahan yang dilakukan:

1. Mengganti for (char c : S) menjadi for (int i = 0; i < len; ++i).
2. Menyimpan S.length() ke dalam variabel len agar program tidak perlu menghitung ulang panjang string pada setiap putaran iterasi (*best practice* untuk efisiensi C++98).
3. Library <algorithm> bisa dihilangkan karena kita tidak lagi membutuhkan fungsi spesifik darinya di versi ini (kita membandingkan nilai maksimum secara manual dengan if (current_len > max_len)).

Penjelasan Baris Program Kasus A :

Berikut adalah penjelasan terstruktur untuk *source code* C++ di atas. Kode ini dirancang untuk bekerja sangat cepat ($\mathcal{O}(N)$) dan kompatibel bahkan hingga versi compiler lawas seperti GCC 3.x (C++98).

1. Pemahaman Strategi Solusi

Format akhir dari string cantik OSN-P selalu terdiri dari dua zona utama:

1. **Zona Kiri:** Berisi kumpulan huruf 'O', 'S', dan 'N' (tiap huruf wajib muncul minimal 1 kali).
2. **Zona Kanan:** dikuasai **sepenuhnya oleh huruf 'P'**, dimulai dari huruf 'P' pertama yang terpilih hingga akhir.

Cara Kerja Algoritma:

Daripada mencoba semua kombinasi penghapusan huruf, kita cukup **memilih satu huruf 'P' pada string S yang akan dijadikan awal Zona Kanan:**

- Semua huruf 'O', 'S', dan 'N' di **sebelah kiri** huruf 'P' tersebut diambil semuanya.
- Semua huruf 'P' dari titik tersebut **ke kanan** diambil semuanya.
- Kita hanya perlu mencatat nilai terbesar (maksimum) dari semua kombinasi posisi 'P' yang valid.

2. Bedah Kode Baris demi Baris

A. Persiapan dan Optimasi Input/Output

```
C++
#include <iostream>
#include <string>
```

```
using namespace std;
```

```
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    • ios_base::sync_with_stdio(false); cin.tie(NULL);: Teknik standar dalam competitive programming C++ untuk mematikan sinkronisasi antara I/O C++ (cin/cout) dan C (scanf/printf). Efeknya, proses membaca string hingga 200.000 karakter menjadi sangat cepat.
```

B. Menghitung Total Huruf 'P' Awal

```
C++
string S;
if (!(cin >> S)) return 0;

int cnt_p_remaining = 0;
int len = S.length();

for (int i = 0; i < len; ++i) {
    if (S[i] == 'P') {
        cnt_p_remaining++;
    }
}
```

- `cnt_p_remaining`: Variabel untuk menyimpan sisa jumlah huruf 'P' yang ada dari indeks saat ini sampai akhir string S .
- **Iterasi 1:** Kita menyapu string S dari awal sampai akhir khusus untuk menghitung berapa banyak huruf 'P' yang dimiliki oleh string tersebut.

C. Mencari Panjang Maksimum dengan Iterasi Utama

```
C++
int cnt_o = 0;
int cnt_s = 0;
int cnt_n = 0;
int max_len = -1;
```

```
for (int i = 0; i < len; ++i) {
    char c = S[i];
```

- cnt_o, cnt_s, cnt_n: Berfungsi sebagai *frekuensi akumulatif* (penghitung) untuk masing-masing huruf 'O', 'S', dan 'N' yang ditemukan di sebelah kiri indeks saat ini.
- max_len = -1: Nilai awal panjang maksimum. Jika hingga akhir iterasi tidak ditemukan kombinasi yang memenuhi syarat, nilai -1 inilah yang akan dicetak (sesuai spesifikasi soal).

D. Logika Saat Menemukan Huruf 'P'

C++

```
if (c == 'P') {
    if (cnt_o > 0 && cnt_s > 0 && cnt_n > 0) {
        int current_len = cnt_o + cnt_s + cnt_n + cnt_p_remaining;
        if (current_len > max_len) {
            max_len = current_len;
        }
    }
    cnt_p_remaining--;
}
```

1. Pengujian Syarat Validitas:

Saat menemukan huruf 'P', anggap huruf ini sebagai **pintu awal Zona Kanan**.

Syaratnya: Zona Kiri wajib punya minimal 1 'O', 1 'S', dan 1 'N' (cnt_o > 0 && cnt_s > 0 && cnt_n > 0).

2. Kalkulasi Panjang (current_len):

- Zona Kiri: cnt_o + cnt_s + cnt_n (semua huruf OSN yang sudah dilewati).
- Zona Kanan: cnt_p_remaining (seluruh huruf 'P' dari posisi saat ini hingga Ujung S).

3. Pembaruan Maksimum:

Jika current_len lebih panjang dari max_len, perbarui max_len.

4. cnt_p_remaining--:

Karena perulangan akan bergerak ke indeks berikutnya (i+1), huruf 'P' pada posisi *i* sekarang berada di "belakang" (sudah dilewati), sehingga sisa huruf 'P' di sebelah kanan berkurang 1.

E. Pencatatan Karakter Non-P dan Output

C++

```
else if (c == 'O') {
    cnt_o++;
}
else if (c == 'S') {
    cnt_s++;
}
else if (c == 'N') {
    cnt_n++;
}
}

cout << max_len << "\n";
return 0;
}
```

- Jika karakter yang ditemui adalah 'O', 'S', atau 'N', kita cukup menambah (*increment*) tabungan jumlah masing-masing huruf tersebut untuk dipakai pada pengujian 'P' berikutnya.
- Di akhir program, cetak max_len.

3. Ringkasan Kompleksitas

Parameter	Kompleksitas	Penjelasan
Waktu (Time)	$\mathcal{O}(N)$	Hanya melakukan 2 kali lewati string sepanjang N ($N \leq 200.000$). Waktu eksekusi ≈ 0.002 detik (jauh di bawah <i>Time Limit</i> 2 detik).
Memori (Space)	$\mathcal{O}(1)$	Hanya memakai beberapa variabel <i>integer</i> sederhana, tidak ada alokasi array/matriks tambahan.

Kompleksitas

- Waktu (Time Complexity):** $\mathcal{O}(N)$, di mana N adalah panjang string S . Program hanya melakukan dua kali proses pembacaan string dari awal hingga akhir (satu untuk mencari 'P', satu lagi untuk iterasi DP). Sangat cepat dan jauh di bawah batas waktu 2 detik.
- Ruang (Space / Memory Complexity):** $\mathcal{O}(1)$ ruang tambahan. Selain string yang disimpan (sekitar ~ 200 KB untuk string 200.000 karakter), program ini hanya memakai variabel primitif bertipe int untuk pencatatan frekuensi (sangat hemat memori, jauh dari limit 256 MB).

Studi Kasus B. Gudang Penyimpanan Kardus

Mari kita bahas satu per satu pertanyaan dari studi kasus **Gudang Penyimpanan Kardus** SONP 2025 ini:

Soal B1

- **Pertanyaan:** Apabila kardus-kardus Pak Dengklek memiliki panjang 17 satuan dan lebar 8 satuan, berapakah paling banyak kardus yang bisa Pak Dengklek simpan di dalam gudang dengan panjang sisi 45 satuan dengan estetik?
- **Penyelesaian:**
 - Ukuran kardus: Panjang $P = 17$, Lebar $L = 8$.
 - Ukuran gudang (persegi): Sisi $S = 45$.
 - Karena kardus harus disusun dengan orientasi yang sama dan sejajar dengan sisi-sisi gudang (tanpa ditumpuk), maka:
 - Banyak kardus yang bisa dimuat dalam satu baris (sepanjang sumbu x) adalah $\left\lfloor \frac{S}{P} \right\rfloor = \left\lfloor \frac{45}{17} \right\rfloor = 2$ kardus ($2 \times 17 = 34 \leq 45$).
 - Banyak kardus yang bisa dimuat dalam satu kolom (sepanjang sumbu y) adalah $\left\lfloor \frac{S}{L} \right\rfloor = \left\lfloor \frac{45}{8} \right\rfloor = 5$ kardus ($5 \times 8 = 40 \leq 45$).
 - Total kardus maksimum yang dapat disimpan adalah hasil kali jumlah baris dan kolom:

$$\text{Total} = 2 \times 5 = 10$$

- **Jawaban B1: 10**

Soal B2

- **Pertanyaan:** BENAR atau SALAH: Apabila Pak Dengklek dapat menyimpan seluruh kardus di dalam gudang dengan panjang sisi S satuan dengan estetik, maka Pak Dengklek pasti dapat menyimpan seluruh kardus di dalam gudang dengan panjang sisi $S + 1$ satuan dengan estetik pula.
- **Penyelesaian:**
 - Jika seluruh kardus muat di dalam gudang berukuran $S \times S$, maka dengan memperbesar ukuran gudang menjadi $(S + 1) \times (S + 1)$, semua ruang yang sebelumnya digunakan tetap tersedia, ditambah ruang ekstra di pinggir.
 - Oleh karena itu, susunan yang sama persis pasti masih bisa dimasukkan ke dalam gudang yang lebih besar $(S + 1)$.
- **Jawaban B2: BENAR**

Soal B3

- **Pertanyaan:** Apabila Pak Dengklek memiliki 31 kardus dengan panjang 3 satuan dan lebar 1 satuan, maka berapa panjang sisi gudang terkecil sehingga Pak Dengklek dapat menyimpan seluruh kardus dengan estetik?
- **Penyelesaian:**
 - Jumlah kardus $N = 31$, dengan ukuran $P = 3, L = 1$.
 - Kita mencari panjang sisi gudang terkecil S sedemikian rupa sehingga kapasitas gudang (yakni jumlah kardus maksimum yang bisa dimuat) ≥ 31 .

- Jika sisi gudang adalah S , maka jumlah maksimum kardus yang dapat ditampung adalah $\left\lfloor \frac{S}{3} \right\rfloor \times \left\lfloor \frac{S}{1} \right\rfloor = \left\lfloor \frac{S}{3} \right\rfloor \times S$.
 - Mari kita coba beberapa nilai integer untuk S :
 - Jika $S = 8$: $\left\lfloor \frac{8}{3} \right\rfloor \times 8 = 2 \times 8 = 16$ (Kurang dari 31)
 - Jika $S = 9$: $\left\lfloor \frac{9}{3} \right\rfloor \times 9 = 3 \times 9 = 27$ (Kurang dari 31)
 - Jika $S = 10$: $\left\lfloor \frac{10}{3} \right\rfloor \times 10 = 3 \times 10 = 30$ (Kurang dari 31)
 - Jika $S = 11$: $\left\lfloor \frac{11}{3} \right\rfloor \times 11 = 3 \times 11 = 33$ (Cukup untuk menampung 31 kardus, karena $33 \geq 31$)
 - Apakah dengan $S = 10$ masih bisa diatur sedemikian rupa? Karena kardus berukuran 3×1 , dalam grid 10×10 kapasitas maksimalnya murni dibatasi oleh perkalian baris $\times$ kolom, yaitu $3 \times 10 = 30$ buah. Jadi $S = 10$ tidak cukup.
 - Sisi terkecil yang memungkinkan adalah 11.
- **Jawaban B3: 11**

Soal Pemrograman Studi Kasus B

Untuk memecahkan masalah ini secara efisien tanpa terkena *Time Limit Exceeded* (TLE) maupun kesalahan ukuran data (*overflow*), kita bisa menggunakan algoritma **Binary Search** (Pencarian Biner).

Pendekatan Logika

1. **Rumus Kapasitas:** Jika kita memiliki gudang dengan panjang sisi S , jumlah maksimum kardus yang bisa ditampung dalam satu baris adalah $\lfloor S/P \rfloor$ dan dalam satu kolom adalah $\lfloor S/L \rfloor$. Total kardus yang bisa disimpan adalah:

$$\text{Total} = \left\lfloor \frac{S}{P} \right\rfloor \times \left\lfloor \frac{S}{L} \right\rfloor$$

2. **Pencarian Biner (Binary Search):** Kita tahu jika sebuah ukuran S cukup untuk N kardus, maka ukuran $S + 1$ pasti juga cukup. Oleh karena sifat monoton (selalu naik) ini, kita bisa mencari S terkecil menggunakan Binary Search, mulai dari $S = 1$ hingga batas aman $S = 3.000.000.000$ (karena batas maksimal secara teoretis ada di sekitar $\sqrt{N \cdot P \cdot L} \approx 2 \times 10^9$).
3. **Mencegah Overflow:** Soal dengan tegas mengingatkan tentang *overflow*. Jika $\lfloor S/P \rfloor$ dan $\lfloor S/L \rfloor$ cukup besar, perkalian keduanya bisa melewati batas maksimal long long di C++. Untuk menghindarinya, alih-alih mengecek:

baris * kolom >= N

Kita menggunakan trik pembagian dengan pembulatan ke atas (ceil):

baris >= (N + kolom - 1) / kolom

Kode Solusi dalam C++

C++

```

#include <iostream>
using namespace std;
int main() {
    // Optimasi input/output pada C++ untuk mempercepat eksekusi
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    long long N, P, L;

    // Membaca masukan
    if (!(cin >> N >> P >> L)) return 0;

    // Batas bawah ukuran S (minimal 1)
    long long low = 1;
    // Batas atas ukuran S yang paling aman (3 Miliar, masuk dalam tipe long long)
    // S maks terjadi saat N = 10^12, P = 2000, L = 2000 -> S = 2.000.000.000
    long long high = 3000000000LL;
    long long ans = high;

    // Algoritma Binary Search
    while (low <= high) {
        long long mid = low + (high - low) / 2;

        long long baris = mid / P;
        long long kolom = mid / L;

        // Jika S terlalu kecil sehingga kardus tidak muat secara dimensi
        if (baris == 0 || kolom == 0) {
            low = mid + 1;
        } else {
            // Mencegah OVERFLOW dengan trik pembagian ceil:
            // baris * kolom >= N identik dengan baris >= ceil(N / kolom)
            if (baris >= (N + kolom - 1) / kolom) {
                ans = mid; // mid cukup besar, simpan sebagai calon jawaban
                high = mid - 1; // coba cari ukuran gudang yang lebih kecil lagi
            } else {
                low = mid + 1; // mid terlalu kecil, harus diperbesar
            }
        }
    }

    // Mencetak output jawaban terkecil
    cout << ans << "\n";

    return 0;
}

```

Kompleksitas

- **Time Complexity:** $O(\log M)$ di mana $M \approx 3 \times 10^9$. while-loop hanya akan berjalan maksimal sekitar ≈ 32 kali. Ini **sangat instan** dan akan lulus *Time Limit* 2 detik dengan amat mudah (0.00x detik).
- **Space Complexity:** $O(1)$. Kita hanya menyimpan variabel primitive dan sama sekali tidak membuat *Array/Vector* raksasa. Eksekusi ini hanya akan memakan KB (KiloBytes) dari total limit 256 MB.

Trik baris $\geq (N + \text{kolom} - 1) / \text{kolom}$ digunakan untuk menggantikan ekspresi baris * kolom $\geq N$ guna menghindari masalah **integer overflow** dan pembulatan pada pembagian integer.

Berikut adalah penjelasan detail mengapa trik ini bekerja dan mengapa sangat penting:

1. Masalah Integer Overflow pada Perkalian

Dalam bahasa C++, setiap tipe data memiliki batas maksimum nilai yang bisa ditampung:

- Tipe long long memiliki batas maksimum sekitar 9×10^{18} (LLONG_MAX).

Pada soal ini, nilai N bisa mencapai 10^{12} , dan nilai S (sisi gudang) bisa mencapai sekitar 3×10^9 .

- Jika $S = 3 \times 10^9$ dan $P = 1$, maka baris = 3×10^9 .
- Jika $L = 1$, maka kolom = 3×10^9 .

Jika kita menghitung baris * kolom:

$$(3 \times 10^9) \times (3 \times 10^9) = 9 \times 10^{18}$$

Angka ini sudah sangat mendekati batas atas long long. Jika S sedikit lebih besar saja (misal saat batas pencarian high diatur lebih besar), hasil perkalian baris * kolom akan melampaui batas 9×10^{18} .

Ketika terjadi **overflow**, nilainya akan memutar menjadi bilangan negatif atau nilai acak yang salah, sehingga kondisi baris * kolom $\geq N$ menghasilkan jawaban yang tidak akurat (bug yang sangat sulit dilacak).

2. Mengapa Mengubah Menjadi Pembagian?

Secara aljabar matematika sederhana:

$$\text{baris} \times \text{kolom} \geq N \Leftrightarrow \text{baris} \geq \frac{N}{\text{kolom}}$$

Dengan mengubahnya menjadi operasi pembagian, nilai yang diolah justru menjadi **semakin kecil**, sehingga tidak ada risiko overflow sama sekali.

3. Mengapa Perlu Ditambah + kolom - 1? (Trik Pembulatan ke Atas / Ceil)

Di dalam pemrograman (termasuk C++), pembagian integer selalu melakukan **pembulatan ke bawah** (truncation/floor).

Contoh masalah jika tanpa + kolom - 1:

- Misalkan $N = 10$ dan kolom = 3.
- Kita ingin tahu berapa baris minimal agar baris $\times 3 \geq 10$.
- Secara matematika: $10/3 = 3.333...$, jadi kita butuh **4 baris** ($4 \times 3 = 12 \geq 10$).
- Namun di C++, hasil dari $10 / 3$ adalah 3 (karena dibulatkan turun).
- Jika kita cek baris $\geq 10 / 3 \rightarrow$ baris ≥ 3 , maka 3 baris dianggap *cukup*, padahal $3 \times 3 = 9 < 10$ (masih kurang!).

Untuk mendapatkan hasil **pembulatan ke atas** ($\lceil N/K \rceil$) hanya menggunakan pembagian integer biasa, rumus umumnya adalah:

$$\left\lceil \frac{A}{B} \right\rceil = \left\lfloor \frac{A + B - 1}{B} \right\rfloor$$

Uji Coba Rumus:

- $N = 10$, kolom = 3
- Pembagian biasa: $10/3 = 3 \rightarrow$ (*salah*)

- Dengan rumus: $(10 + 3 - 1)/3 = 12/3 = 4 \rightarrow \text{(tepat!)}$
- Contoh jika habis dibagi: $N = 9$, kolom = 3
- Dengan rumus: $(9 + 3 - 1)/3 = 11/3 = 3 \rightarrow \text{(tetap tepat!)}$

Kesimpulan

Penggunaan ekspresi:

C++

baris $\geq (N + \text{kolom} - 1) / \text{kolom}$

Secara sempurna menggantikan baris * kolom $\geq N$ karena:

1. **Aman dari Overflow:** Menggunakan operasi pembagian sehingga angkanya mengecil, bukan membesar.
2. **Akurat secara Matematika:** Menghasilkan pembulatan ke atas (*ceil*) yang tepat tanpa perlu mengubah tipe data ke double (yang rentan terhadap masalah presisi desimal).

Studi Kasus C. Balada Kemasan Telur

Berikut adalah pembahasan lengkap untuk soal pemahaman **Studi Kasus C (Jualan Telur)**:

Konsep Dasar & Formula

1. Jumlah Telur yang Dibeli:

- Modal Pak Dengklek = N rupiah.
- Tipe telur yang dipilih = p (di mana p adalah bilangan prima dan p habis membagi N).
- Harga per kemasan $p \times p = p^2$ rupiah.
- Jumlah kemasan yang dibeli = $\frac{N}{p}$ kemasan.
- Karena 1 kemasan berisi $p \times p = p^2$ butir telur, maka:

$$\text{Total Telur} = \frac{N}{p} \times p^2 = N \times p$$

2. Banyak Kemasan $s \times t$ yang Mungkin:

- Banyaknya pasangan (s, t) sehingga $s \times t = \text{Total Telur}$ sama dengan **banyaknya faktor pembagi (divisors)** dari Total Telur.
- Jika bentuk faktorisasi prima dari Total Telur adalah $p_1^{a_1} \times p_2^{a_2} \times \dots \times p_k^{a_k}$, maka jumlah faktor pembaginya adalah:

$$d(\text{Total Telur}) = (a_1 + 1) \times (a_2 + 1) \times \dots \times (a_k + 1)$$

Soal C1

- **Pertanyaan:** Jika $N = 111.111$ dan Pak Dengklek memilih tipe telur $p = 7$, berapa banyak telur yang akan dijual kembali pada akhirnya?
- **Penyelesaian:**
 - Menggunakan rumus $\text{Total Telur} = N \times p$:

$$\text{Total Telur} = 111.111 \times 7 = 777.777$$

- **Jawaban C1: 777777**

Soal C2

- **Pertanyaan:** Dari 5 skenario berikut, manakah banyak telur yang menghasilkan banyak kemungkinan kemasan ($d(\text{Total Telur})$) yang paling banyak?
- **Penyelesaian:**
 - **Pilihan A:** $2^1 \times 5^1 = 10 \Rightarrow d(10) = (1 + 1)(1 + 1) = 4$
 - **Pilihan B:** $2^2 \times 5^2 = 100 \Rightarrow d(100) = (2 + 1)(2 + 1) = 9$
 - **Pilihan C:** $2^2 \times 5^1 = 20 \Rightarrow d(20) = (2 + 1)(1 + 1) = 6$
 - **Pilihan D:** $2^3 \times 5^1 = 40 \Rightarrow d(40) = (3 + 1)(1 + 1) = 8$
 - **Pilihan E:** $2^6 \times 5^0 = 64 \Rightarrow d(64) = (6 + 1) = 7$

Jumlah faktor terbesar diantara pilihan tersebut adalah **9** (Pilihan B).

- **Jawaban C2: B**

Soal C3

- **Pertanyaan:** Jika modal $N = 45$ rupiah, tipe telur p manakah yang harus dipilih agar jumlah kemungkinan kemasan sebanyak-banyaknya? (Jika ada lebih dari satu, pilih p terkecil).

- **Penyelesaian:**

- Faktorisasi prima dari $N = 45$ adalah $3^2 \times 5$.
- Faktor prima dari 45 (tipe telur p yang mungkin dipilih) adalah **3** dan **5**.

1. **Jika memilih $p = 3$:**

$$\text{Total Telur} = N \times p = 45 \times 3 = 135$$

- Faktorisasi prima dari $135 = 3^3 \times 5^1$.
- Banyak kemungkinan kemasan $d(135) = (3 + 1) \times (1 + 1) = 4 \times 2 = 8$.

2. **Jika memilih $p = 5$:**

$$\text{Total Telur} = N \times p = 45 \times 5 = 225$$

- Faktorisasi prima dari $225 = 3^2 \times 5^2$.
- Banyak kemungkinan kemasan $d(225) = (2 + 1) \times (2 + 1) = 3 \times 3 = 9$.

Nilai $d(\text{Total Telur})$ terbanyak diperoleh saat memilih $p = 5$ (yaitu 9 kemasan).

- **Jawaban C3: 5**

Soal Pemograman

Untuk menyelesaikan masalah ini secara efisien dengan batas waktu 2 detik dan nilai $N \leq 10^{12}$, kita perlu menggunakan pendekatan matematis berdasarkan **Teori Bilangan (Faktorisasi Prima)** alih-alih mencoba semua kemungkinan ukuran kemasan satu per satu.

Pendekatan Logika & Matematika

1. **Faktorisasi Prima:**

Setiap bilangan N dapat difaktorkan menjadi perkalian bilangan prima:

$$N = p_1^{a_1} \times p_2^{a_2} \times \dots \times p_k^{a_k}$$

2. **Menghitung Kemungkinan Kemasan (Faktor Pembagi):**

Rumus untuk mencari banyak pembagi dari N adalah:

$$D(N) = (a_1 + 1) \times (a_2 + 1) \times \dots \times (a_k + 1)$$

3. **Menganalisis Pilihan Telur:**

Jika Pak Dengklek memilih tipe telur p_i (di mana p_i pasti salah satu dari faktor prima N), maka jumlah telurnya menjadi $N \times p_i$.

Ini berarti pangkat dari p_i pada faktorisasi prima bertambah 1 (dari a_i menjadi $a_i + 1$).

Banyaknya kemasan baru akan menjadi:

$$D(N \times p_i) = D(N) \times \frac{a_i + 2}{a_i + 1}$$

4. **Kunci Jawaban:**

Kita ingin memaksimalkan $D(N \times p_i)$. Karena $D(N)$ adalah angka tetap, kita hanya perlu **memaksimalkan pengali $\frac{a_i+2}{a_i+1}$** .

Fungsi pecahan ini nilainya akan semakin besar jika a_i **sekecil mungkin**.

Oleh karena itu, strateginya sangat sederhana:

- Cari faktorisasi prima dari N .
- Temukan faktor prima dengan **pangkat (a_i) yang paling kecil**.
- Jika ada beberapa faktor prima dengan pangkat terkecil yang sama, pilih **bilangan prima yang paling kecil** untuk memenuhi syarat pada soal ("*pilih yang paling kecil*").

Kode Solusi dalam C++

C++

```
#include <iostream>
using namespace std;

int main() {
    // Optimasi input/output pada C++
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    long long N;
    if (!(cin >> N)) return 0;

    long long min_pangkat = 1e18; // Inisialisasi dengan nilai yang sangat besar
    long long best_p = -1;

    // Faktorisasi prima N dari 2 hingga sqrt(N)
    for (long long i = 2; i * i <= N; i++) {
        if (N % i == 0) {
            long long pangkat = 0;
            // Hitung ada berapa pangkat untuk faktor prima i
            while (N % i == 0) {
                pangkat++;
                N /= i;
            }

            // Jika pangkat yang ditemukan lebih kecil dari rekor minimum saat ini
            if (pangkat < min_pangkat) {
                min_pangkat = pangkat;
                best_p = i;
            }
        }
    }

    // Jika N masih bersisa lebih dari 1, maka sisa N tersebut adalah bilangan prima
    // Pangkatnya pasti tepat 1.
    if (N > 1) {
        if (1 < min_pangkat) {
            best_p = N;
        }
        // Catatan: Jika min_pangkat sudah sama dengan 1 dari loop sebelumnya,
        // kita tidak menimpa best_p karena kita harus memilih P yang paling kecil.
        // Faktor yang didapat di dalam loop pasti lebih kecil dari sisa N.
    }

    // Cetak tipe telur (faktor prima) yang terpilih
    cout << best_p << "\n";

    return 0;
}
```


}

Kompleksitas

- **Time Complexity:** $O(\sqrt{N})$. Loop iterasi maksimal akan berjalan hingga batas $\sqrt{10^{12}} = 1.000.000$ (1 Juta) operasi. Dalam C++, 1 juta operasi membutuhkan waktu eksekusi kurang dari ≈ 0.01 detik. Ini dengan santai melewati *Time Limit* 2 detik dari Subsoal 2 (Sulit).
- **Space/Memory Complexity:** $O(1)$. Program tidak menggunakan array sehingga pemakaian memori kurang dari 1 MB dan tidak akan melanggar *Memory Limit* 256 MB.

Studi Kasus D. Telur Kemasan Balado

Deskripsi Cerita

Berikut adalah pembahasan dan jawaban lengkap untuk soal pemahaman **Studi Kasus D (Telur Kemasan Balado)**:

Konsep Dasar & Formula

1. Aturan Operasi Penyembunyian:

- Setiap kali melakukan 1 operasi, Pak Dengklek menghapus sebuah rentang $[l, r]$ secara penuh.
- Karena seluruh telur yang tersisa **harus varian Balado ('B')**, maka setiap bagian yang tersimpan harus berupa blok telur 'B' (kontigu).

2. Menghitung Banyaknya Operasi yang Dibutuhkan:

Jika kita memilih untuk mempertahankan m buah blok telur 'B' ($B_{i_1}, B_{i_2}, \dots, B_{i_m}$):

- Menghapus bagian di antara blok-blok yang disimpan membutuhkan $m - 1$ operasi.
- Menghapus bagian sebelum blok pertama (jika string tidak dimulai dari blok tersebut) membutuhkan **1 operasi**.
- Menghapus bagian setelah blok terakhir (jika string tidak diakhiri oleh blok tersebut) membutuhkan **1 operasi**.

$$\text{Total Operasi} = (\text{start} > 1? 1: 0) + (m - 1) + (\text{end} < N? 1: 0)$$

Soal D1

- **Pertanyaan:** Apabila Pak Dengklek hanya boleh melakukan operasi penyembunyian paling banyak **1 kali** ($K = 1$), manakah pilihan string S yang menghasilkan telur varian Balado terbanyak?
- **Pilihan & Analisis** ($K = 1$):
 - **A. "BBBBB"**
 - Tidak perlu dihapus (0 operasi).
 - Total Balado = **5**.
 - **B. "BBAABBBB"**
 - Blok 'B': $B_1 = \text{"BB"}$ (panjang 2, posisi 1..2), $B_2 = \text{"BBBB"}$ (panjang 4, posisi 5..8).
 - Dengan 1 operasi, kita bisa menghapus "AA" di tengah (posisi 3..4).
 - Karena B_1 di awal (posisi 1) dan B_2 di akhir (posisi 8), total operasi = $0 + 1 + 0 = 1$.
 - Total Balado = $2 + 4 = 6$.
 - **C. "AAAAABBBB"**
 - Hapus "AAAAA" di depan (1 operasi).
 - Total Balado = **4**.
 - **D. "BABBBBBBBBAB"**
 - Dengan 1 operasi, hapus "A BBBBBBBB A" di tengah (posisi 2..11) menyisakan "BB" → **2** Balado.
 - **E. "AABBBBAABBBBABBB"**

- Dengan 1 operasi, hapus posisi 1..14 menyisakan blok terakhir "BBB" → 3 Balado.

- **Jawaban D1: B**

Soal D2

- **Pertanyaan:** Untuk string $S = \text{"BAABBBAAAABBBBAAABBA"}$ dengan paling banyak **2 kali** operasi ($K = 2$), berapa telur Balado terbanyak yang dapat diperoleh?
- **Analisis String:**
 - $B_1 = \text{"B"}$ (panjang 1, posisi 1)
 - $B_2 = \text{"BBB"}$ (panjang 3, posisi 4..6)
 - $B_3 = \text{"BBBB"}$ (panjang 4, posisi 11..14)
 - $B_4 = \text{"BB"}$ (panjang 2, posisi 18..19)
 - Panjang total $N = 20$.
- **Mencoba Kombinasi Blok dengan $K \leq 2$ Operasi:**
 - Simpan B_1 dan B_3 :
 - Op 1: Hapus posisi 2..10 (AA BBB AAAA).
 - Op 2: Hapus posisi 15..20 (AAA BB A).
 - Total operasi = $0 + 1 + 1 = 2 \leq 2$.
 - Total Balado = $1 + 4 = 5$.
 - (Kombinasi lain yang menghasilkan lebih dari 5 membutuhkan minimal 3 operasi).

- **Jawaban D2: 5**

Soal D3

- **Pertanyaan:** Untuk string $S = \text{"BAABBBAAAABBBBAAABBA"}$ dengan paling banyak **3 kali** operasi ($K = 3$), berapa telur Balado terbanyak yang dapat diperoleh?
- **Mencoba Kombinasi Blok dengan $K \leq 3$ Operasi:**
 - Simpan B_1 , B_2 , dan B_3 :
 - Op 1: Hapus posisi 2..3 (AA).
 - Op 2: Hapus posisi 7..10 (AAAA).
 - Op 3: Hapus posisi 15..20 (AAA BB A).
 - Total operasi = $0 + 2 + 1 = 3 \leq 3$.
 - Total Balado = $1 + 3 + 4 = 8$.
- **Jawaban D3: 8**

Soal Pemograman

Berikut adalah pembahasan dan solusi kode lengkap dalam bahasa C++ untuk **Soal Pemrograman D (Telur Kemasan Balado)**.

Pendekatan Logika & Algoritma

Tujuan kita adalah menyisakan sebanyak mungkin telur varian Balado ('B') dengan menghapus telur varian Asli ('A') dalam maksimal K operasi. Karena satu operasi menghapus sebuah

rentang $[l, r]$, operasi ini sangat fleksibel dan pada dasarnya mengizinkan kita menghapus *seluruh gap/jeda* (yang berisi 'A') di antara dua blok 'B' sekaligus.

1. Memecah String Menjadi Blok-blok 'B':

Pertama, abaikan semua 'A' dan catat panjang setiap blok kontigu (berderet) yang berisi 'B'.

Misalkan dari string kita mendapatkan M buah blok 'B' dengan panjang masing-masing $L_1, L_2, \dots, L_M$.

2. Menghitung Biaya Operasi (Cost):

Jika kita ingin mempertahankan k buah blok 'B' dan menggabungkannya, kita butuh:

- $k - 1$ **operasi** untuk menghapus semua jeda di antara k blok 'B' yang kita pilih.
- **1 operasi tambahan** jika kita perlu menghapus awalan (prefix) yang ada sebelum blok 'B' pertama pilihan kita. (Terkecuali blok pertama yang kita pilih adalah blok terujung kiri string, dan string itu memang diawali oleh 'B').
- **1 operasi tambahan** jika kita perlu menghapus akhiran (suffix) yang ada sesudah blok 'B' terakhir pilihan kita. (Terkecuali blok terakhir yang kita pilih adalah blok terujung kanan string, dan string itu memang diakhiri oleh 'B').

3. Menganalisis 4 Skenario Berdasarkan Batas Tepi (Boundary):

Dengan membagi ke dalam 4 strategi batasan tepi, secara matematis, jumlah maksimal **tambahan blok 'B' dari tengah** yang bisa kita pilih setelah blok-blok di ujung ternyata secara konstan dibatasi oleh $K - 1$.

- **Skenario A:** Tidak memprioritaskan blok terujung manapun. Kita bebas mengambil hingga maksimal $K - 1$ blok terbesar dari keseluruhan blok.
- **Skenario B:** Mempertahankan blok terujung kiri (hanya valid jika string diawali 'B'). Kita mengambil blok pertama, ditambah bebas memilih hingga maksimal $K - 1$ blok terbesar dari sisanya.
- **Skenario C:** Mempertahankan blok terujung kanan (hanya valid jika string diakhiri 'B'). Kita mengambil blok terakhir, ditambah bebas memilih hingga maksimal $K - 1$ blok terbesar dari sisanya.
- **Skenario D:** Mempertahankan blok terujung kiri sekaligus kanan (hanya valid jika string diawali dan diakhiri 'B'). Kita mengambil blok pertama dan terakhir, ditambah bebas memilih hingga maksimal $K - 1$ blok terbesar dari sisanya.

Kita cukup mensimulasikan keempat skenario (jika valid), mengambil total jumlah 'B' dari masing-masing, dan mencari **nilai terbesarnya (maksimum)**.

Kode Solusi dalam C++

C++

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>

using namespace std;

// Fungsi bantuan untuk mengambil jumlah telur maksimal dari sisa blok yang ada
long long get_max(long long forced_sum, long long allowed, vector<long long>& avail) {
    if (allowed < 0) return 0; // Jika jumlah operasi tidak cukup, skenario ini tidak valid

    long long sum = forced_sum;

    // Sortir blok sisa secara menurun (descending) untuk mendapatkan blok terpanjang
```

```

sort(avail.rbegin(), avail.rend());

// Ambil maksimal `allowed` blok terbesar (atau semua jika allowed melebihi ukuran)
long long take = min(allowed, (long long)avail.size());
for (long long i = 0; i < take; i++) {
    sum += avail[i];
}

return sum;
}

int main() {
    // Optimasi Input/Output
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N;
    long long K;
    if (!(cin >> N >> K)) return 0;

    string S;
    cin >> S;

    // Kumpulkan panjang semua blok 'B' yang berurutan
    vector<long long> L;
    long long current_len = 0;
    for (int i = 0; i < N; i++) {
        if (S[i] == 'B') {
            current_len++;
        } else {
            if (current_len > 0) {
                L.push_back(current_len);
                current_len = 0;
            }
        }
    }
    if (current_len > 0) {
        L.push_back(current_len);
    }

    int M = L.size();

    // Kasus Dasar: Jika tidak ada varian Balado sama sekali
    if (M == 0) {
        cout << 0 << "\n";
        return 0;
    }

    // Cek apakah string murni diawali atau diakhiri dengan 'B'
    bool has_prefix = (S[0] == 'B');
    bool has_suffix = (S[N - 1] == 'B');

    long long ans = 0;
    long long allowed = K - 1; // Formula konstan untuk operasi ekstra

    // -----
    // Skenario A: Tanpa mengasumsikan bebas dari tepi mana pun
    // -----
    if (allowed >= 0) {
        vector<long long> avail = L;
        ans = max(ans, get_max(0, allowed, avail));
    }

```

```

}

// -----
// Skenario B: Mengasumsikan blok pertama bebas biaya prefix
// -----
if (has_prefix && M >= 1 && allowed >= 0) {
    vector<long long> avail;
    for (int i = 1; i < M; i++) avail.push_back(L[i]);
    ans = max(ans, get_max(L[0], allowed, avail));
}

// -----
// Skenario C: Mengasumsikan blok terakhir bebas biaya suffix
// -----
if (has_suffix && M >= 1 && allowed >= 0) {
    vector<long long> avail;
    for (int i = 0; i < M - 1; i++) avail.push_back(L[i]);
    ans = max(ans, get_max(L[M - 1], allowed, avail));
}

// -----
// Skenario D: Blok pertama & terakhir bebas biaya (hanya jeda)
// -----
if (has_prefix && has_suffix && M >= 1) {
    if (M == 1) {
        // Jika seluruh string dipenuhi 'B', tidak ada jeda yang perlu dihapus (Cost 0)
        ans = max(ans, L[0]);
    } else {
        if (allowed >= 0) {
            vector<long long> avail;
            for (int i = 1; i < M - 1; i++) avail.push_back(L[i]);
            ans = max(ans, get_max(L[0] + L[M - 1], allowed, avail));
        }
    }
}

// Tampilkan hasil optimal dari keempat skenario
cout << ans << "\n";

return 0;
}

```

Studi Kasus E. Tebas Tebang Bambu

Berikut adalah pembahasan dan jawaban lengkap untuk **Studi Kasus E (TebasTebang Bambu)**:

Aturan Dasar & Syarat

1. **Seluruh N bambu harus terjual** (dimasukkan ke dalam ikat).
2. Setiap ikat terdiri dari **minimal M bilah bambu**.
3. Bambu dalam **satu ikat yang sama harus berpanjang sama**.
4. Panjang asli A_i hanya bisa dipangkas sebanyak $0 \leq x \leq K$ satuan (panjang baru $A_i' \in [A_i - K, A_i]$).
5. **Setiap bambu tidak boleh dibelah** menjadi beberapa bilah.

Soal E1

- **Pertanyaan:** Jika $K = 0$ (tidak bisa dipangkas) dan $M = 3$, skenario mana yang **memungkinkan seluruh 14 bambu terjual** dan menghasilkan **banyaknya ikat paling banyak**?
- **Analisis Skenario ($K = 0, M = 3$):**

Agar seluruh bambu terjual, frekuensi setiap panjang bambu yang ada harus bisa dibagi habis atau dikelompokkan ke dalam ikat-ikat berisi minimal 3 bilah.

- **Pilihan A:** [1,2,3,4,5,6,7,7,6,5,4,3,2,1]
 - Frekuensi: Panjang 1 ada 2, Panjang 2 ada 2, dst.
 - Karena frekuensi masing-masing panjang < 3 , **tidak bisa dijual seluruhnya**.
 - **Pilihan B:** [2,3,4,2,3,4,2,3,4,2,3,4,2,3]
 - Frekuensi: Panjang 2 (5 bilah), Panjang 3 (5 bilah), Panjang 4 (4 bilah).
 - Panjang 2: 5 bilah $\rightarrow$ bisa dibuat 1 ikat (isi 5) ✓.
 - Panjang 3: 5 bilah $\rightarrow$ bisa dibuat 1 ikat (isi 5) ✓.
 - Panjang 4: 4 bilah $\rightarrow$ bisa dibuat 1 ikat (isi 4) ✓.
 - Total ikat = **3 ikat** (seluruh 14 bambu terjual).
 - **Pilihan C:** [5,5,5,5,6,6,6,6,7,7,7,7,8,8]
 - Panjang 8 hanya ada 2 bilah (< 3). **Tidak bisa dijual seluruhnya**.
 - **Pilihan D:** [8,7,8,7,8,7,8,7,8,7,8,7,8,7]
 - Frekuensi: Panjang 8 (7 bilah), Panjang 7 (7 bilah).
 - Panjang 8: 7 bilah $\rightarrow$ bisa buat 2 ikat (isi 3 dan 4) atau 1 ikat (isi 7).
 - Panjang 7: 7 bilah $\rightarrow$ bisa buat 2 ikat (isi 3 dan 4) atau 1 ikat (isi 7).
 - Total ikat terbanyak = $2 + 2 = 4$ ikat (seluruh 14 bambu terjual).
 - **Pilihan E:** [1,1,1,2,3,3,3,4,5,5,5,6,6,6]
 - Panjang 2 ada 1 bilah, Panjang 4 ada 1 bilah (< 3). **Tidak bisa dijual seluruhnya**.
- **Jawaban E1: D** (Membentuk 4 ikat, lebih banyak dari pilihan B yang hanya 3 ikat).

Soal E2

- **Pertanyaan:** BENAR atau SALAH: Pada kasus-kasus yang memungkinkan Pak Dengklek untuk menjual seluruh bambu, maka untuk memaksimumkan banyaknya ikat bambu yang dapat Pak Dengklek jual, setiap ikat bambu pasti terdiri dari kurang dari $2 \times M$ bilah bambu.
- **Penyelesaian:**
 - Misalkan sebuah ikat berisi X bilah bambu dengan $X \geq 2 \times M$.
 - Karena seluruh X bilah bambu ini memiliki panjang yang sama, kita bisa memisahkan/memecah ikat tunggal ini menjadi **2 ikat terpisah** (misalnya ikat 1 berisi M bilah, dan ikat 2 berisi $X - M \geq M$ bilah).
 - Pecahan 2 ikat tersebut tetap memenuhi syarat bahwa setiap ikat berisi $\geq M$ bilah dan berpanjang sama.
 - Hasil pemecahan ini **meningkatkan jumlah total ikat**. Oleh karena itu, dalam susunan yang memaksimumkan jumlah ikat, tidak pernah ada ikat yang jumlah bilahnya $\geq 2 \times M$. Dengan kata lain, semua ikat harus terdiri dari $< 2 \times M$ bilah.
- **Jawaban E2: BENAR**

Soal E3

- **Pertanyaan:**
 - Bambu (18 bilah): [1,1,3,3,4,5,6,7,8,9,9,13,13,14,14,14,15,16]
 - Maksimal pemangkasan $K = 2$, minimal bilah per ikat $M = 3$.
 - Harus menjual **seluruh 18 bambu**. Berapa ikat maksimum yang dapat dijual?
- **Penyelesaian:**

Setiap bambu A_i dapat dipangkas menjadi rentang $[A_i - 2, A_i]$.

Kita harus mengelompokkan ke-18 bambu ini menjadi grup-grup beranggota ≥ 3 sedemikian rupa sehingga rentang pemangkasan seluruh bambu di dalam satu grup memiliki **irisan (overlap)** panjang target yang sama.

Mari kita urutkan dan analisis dari bambu terkecil:

1. **Bambu 1–4:** [1,1,3,3]
 - Dipangkas ke rentang: $1 \in [1,1]$, $3 \in [1,3]$.
 - Keempat bambu ini bisa dijadikan **1 ikat berpanjang 1** (karena $3 - 2 = 1$).
 - Terdiri dari **4 bilah** (≥ 3).
2. **Bambu 5–11:** [4,5,6,7,8,9,9] (7 bilah)
 - Bambu ke-5 s.d. 8 ([4,5,6,7]) dipangkas ke rentang target panjang 4 or 5:
 - $4 \in [2,4]$, $5 \in [3,5]$, $6 \in [4,6]$.
 - Bisakah dibentuk 2 ikat dari 7 bilah ini?
 - **Ikat I (3 bilah):** [4,5,6] $\rightarrow$ bisa dipangkas jadi **panjang 4**.
 - **Ikat II (4 bilah):** [7,8,9,9] $\rightarrow$ bisa dipangkas jadi **panjang 7** ($9 - 2 = 7 \leq 7$).
 - Jadi 7 bilah ini dapat membentuk **2 ikat** (masing-masing 3 dan 4 bilah).
3. **Bambu 12–18:** [13,13,14,14,14,15,16] (7 bilah)
 - Bisakah dibentuk 2 ikat dari 7 bilah ini?

- **Ikat III (3 bilah):** [13,13,14] → bisa dipangkas jadi **panjang 13**.
- **Ikat IV (4 bilah):** [14,14,15,16] → bisa dipangkas jadi **panjang 14** ($16 - 2 = 14$).
- Jadi 7 bilah ini dapat membentuk **2 ikat** (masing-masing 3 dan 4 bilah).
- **Total Ikat:** $1 + 2 + 2 = 5$ ikat.
- Seluruh 18 bambu berhasil dikelompokkan tanpa ada yang tersisa.
- **Jawaban E3: 5**

Soal Pemograman E:

Berikut adalah pembahasan dan solusi kode lengkap dalam bahasa C++ untuk **Soal Pemrograman E (TebasTebang Bambu)**.

Pendekatan Logika & Algoritma (Dynamic Programming)

Masalah ini meminta kita untuk mengelompokkan (mempartisi) seluruh bilah bambu menjadi kelompok-kelompok (ikat) sedemikian rupa sehingga:

1. Setiap kelompok memiliki setidaknya M elemen.
2. Selisih antara elemen terpanjang dan terpendek di setiap kelompok maksimal K (karena bambu panjang A_{max} hanya bisa dipangkas maksimal sebanyak K agar bisa sama dengan A_{min}).

Untuk memaksimalkan jumlah grup (ikat), kita dapat menggunakan **Dynamic Programming (DP)** yang dioptimasi.

- **Langkah 1 (Sorting):** Urutkan panjang bambu A secara membesar (*ascending*). Dengan pengurutan ini, bambu-bambu yang masuk ke dalam satu ikat pasti merupakan elemen-elemen yang bersebelahan (sub-array kontinu).
- **Langkah 2 (Definisi DP):** Misalkan $dp[i]$ adalah *jumlah maksimal ikat bambu yang bisa dibentuk dari bambu ke-1 hingga bambu ke- i* .
- **Langkah 3 (Transisi):**

Jika kita menjadikan bambu indeks ke- $(j + 1)$ hingga ke- i sebagai satu ikat baru, maka:

$$dp[i] = \max(dp[j]) + 1$$

Dengan syarat untuk j :

- $j \leq i - M$ (agar ikat yang baru punya ukuran minimal M).
- $A[i] - A[j + 1] \leq K$ (selisih bambu terpanjang dan terpendek di ikat ini maksimal K).
- Kondisi kedua bisa diubah menjadi $A[j + 1] \geq A[i] - K$.
- **Langkah 4 (Optimasi):**
 - Untuk mencari syarat $A[j + 1] \geq A[i] - K$, kita bisa mencari batas minimum indeks dengan **Binary Search** (`std::lower_bound`). Misal ketemu batas minimum di indeks L . Maka, rentang nilai j yang valid adalah $[L - 1, i - M]$.
 - Kita perlu mencari nilai maksimum $dp[j]$ di dalam rentang tersebut secara cepat. Untuk itu, kita bisa menggunakan struktur data **Segment Tree** atau **Monotonic Queue**, yang akan menjawab nilai maksimal di rentang tersebut dalam waktu $O(\log N)$ atau $O(1)$.

Kode Solusi dalam C++ (Menggunakan Segment Tree)

C++

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

const int INF = 1e9;
vector<int> tree_max;

// Fungsi untuk memperbarui Segment Tree (Point Update)
void update(int node, int l, int r, int idx, int val) {
    if (l == r) {
        tree_max[node] = val;
        return;
    }
    int mid = l + (r - l) / 2;
    if (idx <= mid) {
        update(2 * node, l, mid, idx, val);
    } else {
        update(2 * node + 1, mid + 1, r, idx, val);
    }
    tree_max[node] = max(tree_max[2 * node], tree_max[2 * node + 1]);
}

// Fungsi untuk mencari nilai DP terbesar pada rentang (Range Max Query)
int query(int node, int l, int r, int ql, int qr) {
    if (ql > r || qr < l) return -INF;
    if (ql <= l && r <= qr) return tree_max[node];

    int mid = l + (r - l) / 2;
    return max(query(2 * node, l, mid, ql, qr),
               query(2 * node + 1, mid + 1, r, ql, qr));
}

int main() {
    // Optimasi Input/Output agar tidak Time Limit Exceeded (TLE)
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N, M;
    long long K;
    if (!(cin >> N >> M >> K)) return 0;

    // Membaca array dari indeks 1 (1-based index)
    vector<long long> A(N + 1);
    for (int i = 1; i <= N; ++i) {
        cin >> A[i];
    }

    // Mengurutkan bambu agar pengelompokan sub-array optimal
    sort(A.begin() + 1, A.end());

    // Inisialisasi Segment Tree dan array DP dengan nilai -INF (tidak mungkin dicapai)
    tree_max.assign(4 * (N + 1), -INF);
    vector<int> dp(N + 1, -INF);

    // Base case: 0 bambu berarti 0 ikat
    dp[0] = 0;
```

```

update(1, 0, N, 0, 0);

for (int i = 1; i <= N; ++i) {
    // Binary search untuk mencari indeks j+1 terkecil dimana A[j+1] >= A[i] - K
    int L_i = lower_bound(A.begin() + 1, A.begin() + i + 1, A[i] - K) - A.begin();

    // Rentang nilai 'j' yang valid untuk menjadi ujung partisi sebelumnya
    int left_bound = L_i - 1;
    int right_bound = i - M;

    // Jika rentang batas valid, cari nilai DP terbesar di antara rentang tersebut
    if (left_bound <= right_bound && right_bound >= 0) {
        int max_prev = query(1, 0, N, left_bound, right_bound);

        // Jika valid (ada cara menyusun sebelumnya), maka ikat bertambah 1
        if (max_prev != -INF) {
            dp[i] = max_prev + 1;
        }
    }

    // Masukkan state dp[i] saat ini ke dalam Segment Tree agar bisa di-query
    kedepannya
    if (dp[i] != -INF) {
        update(1, 0, N, i, dp[i]);
    }
}

// Output -1 jika tidak memungkinkan seluruh bambu dipartisi, jika mungkin,
keluarkan dp[N]
if (dp[N] == -INF) {
    cout << -1 << "\n";
} else {
    cout << dp[N] << "\n";
}

return 0;
}

```

Analisis Kompleksitas

- **Waktu Eksekusi (Time Complexity):** $O(N \log N)$. Terdapat $O(N \log N)$ untuk melakukan *sorting* di awal. Setelahnya, di dalam *loop* berjalan hingga N , terdapat *lower_bound* $O(\log N)$ dan *Segment Tree query/update* sebesar $O(\log N)$. Kode ini akan selesai dieksekusi murni di bawah 0.05 detik (sangat aman dari limitasi 3.0 detik).
- **Pemakaian Memori (Space Complexity):** $O(N)$ untuk menyimpan array bambu A , array dp , dan struktur pohon *Segment Tree* $tree_max$. Total memori berada pada kisaran ~ 5 MB, yang jauh sekali di bawah *memory limit* 256 MB.

Pembahasan Source Code :

Penjelasan fungsi :

```

// Fungsi untuk memperbarui Segment Tree (Point Update)
void update(int node, int l, int r, int idx, int val) {
    if (l == r) {
        tree_max[node] = val;
        return;
    }
    int mid = l + (r - l) / 2;
    if (idx <= mid) {
        update(2 * node, l, mid, idx, val);
    }
}

```

```

    } else {
        update(2 * node + 1, mid + 1, r, idx, val);
    }
    tree_max[node] = max(tree_max[2 * node], tree_max[2 * node + 1]);
}

```

Fungsi update ini digunakan untuk memperbaiki satu elemen (*Point Update*) pada **Segment Tree**. Ketika nilai suatu elemen pada indeks tertentu di array asli diubah, fungsi ini akan memperbaiki nilai tersebut di daun (*leaf node*) Segment Tree, lalu memperbaiki nilai maksimum di semua node induknya yang terpengaruh.

Kompleksitas waktunya sangat efisien, yaitu $O(\log N)$.

Parameter Fungsi

Parameter	Arti / Fungsi
node	Indeks node saat ini di dalam array tree_max.
l, r	Batas rentang yang diwakili oleh node saat ini (l sampai r).
idx	Indeks pada array asli yang nilainya ingin diubah.
val	Nilai baru yang akan dimasukkan ke idx.

Cara Kerja Kode (Baris demi Baris)

1. Kasus Basis / Node Daun (*Base Case*)

C++

```

if (l == r) {
    tree_max[node] = val;
    return;
}

```

- **Artinya:** Jika $l == r$, berarti pencarian sudah sampai di node daun (*leaf node*) yang secara spesifik mewakili elemen pada indeks idx.
- **Aksi:** Nilai di node tersebut langsung diubah menjadi val, lalu fungsi selesai (return).

2. Penelusuran Kebawah (*Traversal Down*)

C++

```

int mid = l + (r - l) / 2;
if (idx <= mid) {
    update(2 * node, l, mid, idx, val);
} else {
    update(2 * node + 1, mid + 1, r, idx, val);
}

```

- Jika belum sampai di daun, fungsi menentukan apakah idx berada di sebelah kiri atau kanan titik tengah (mid):

- **Jika** $idx \leq mid$: Masuk ke anak kiri ($2 * node$) yang mencakup rentang l hingga mid .
 - **Jika** $idx > mid$: Masuk ke anak kanan ($2 * node + 1$) yang mencakup rentang $mid + 1$ hingga r .
- Hanya **satu cabang** yang dikunjungi, bukan keduanya. Inilah yang membuat prosesnya berjalan cepat $O(\log N)$.

3. Pembaruan Nilai Induk (*Backtracking / Pull Up*)

C++

```
tree_max[node] = max(tree_max[2 * node], tree_max[2 * node + 1]);
```

- Baris ini dieksekusi **setelah** rekursi anak selesai (*saat fungsi kembali ke atas*).
- Node induk memperbarui nilainya sendiri dengan mengambil nilai maksimum dari kedua anak yang ada di bawahnya ($2 * node$ dan $2 * node + 1$).

Ringkasan Alur Kerja

1. **Turun ke bawah:** Mencari node daun yang sesuai dengan idx .
2. **Ubah nilai:** Mengganti nilai di node daun tersebut menjadi val .
3. **Naik ke atas:** Memperbarui nilai maksimum pada tiap node di sepanjang jalur kembali ke akar (*root*).

Penjelasan fungsi :

```
int query(int node, int l, int r, int ql, int qr) {
    if (ql > r || qr < l) return -INF;
    if (ql <= l && r <= qr) return tree_max[node];

    int mid = l + (r - l) / 2;
    return max(query(2 * node, l, mid, ql, qr),
               query(2 * node + 1, mid + 1, r, ql, qr));
}
```

Fungsi query di atas adalah bagian dari struktur data **Segment Tree** (pohon segmen). Fungsi ini digunakan untuk mencari nilai **maksimum (Range Maximum Query - RMQ)** dalam rentang (*range*) tertentu pada sebuah array dengan kompleksitas waktu yang sangat cepat, yaitu $O(\log N)$.

Mari kita bedah setiap bagian kode dan parameternya secara mudah.

Parameter Fungsi

Parameter	Arti / Fungsi
node	Indeks node saat ini di dalam array tree_max.
l, r	Batas cakupan node saat ini: l (left) sampai r (right).
ql, qr	Rentang yang ditanyakan oleh user: ql (query left) sampai qr (query right).

Cara Kerja Kode (Baris demi Baris)

1. Kasus Tidak Irisan (*No Overlap*)

C++

```
if (ql > r || qr < l) return -INF;
```

- **Artinya:** Rentang node saat ini (l sampai r) berada sepenuhnya **di luar** rentang yang dicari (ql sampai qr).
- **Aksi:** Mengembalikan nilai -INF (negatif tak hingga) agar nilai dari node ini diabaikan saat dibandingkan dengan fungsi max().

2. Kasus Total Irisan (*Total Overlap*)

C++

```
if (ql <= l && r <= qr) return tree_max[node];
```

- **Artinya:** Rentang node saat ini (l sampai r) berada sepenuhnya **di dalam** rentang yang dicari (ql sampai qr).
- **Aksi:** Tidak perlu mencari lebih jauh ke bawah. Langsung kembalikan nilai maksimum yang tersimpan di tree_max[node].

3. Kasus Irisan Parsial (*Partial Overlap*)

C++

```
int mid = l + (r - l) / 2;
```

```
return max(query(2 * node, l, mid, ql, qr),
```

```
query(2 * node + 1, mid + 1, r, ql, qr));
```

- Jika rentang node saat ini hanya **sebagian** masuk ke dalam rentang query, maka pencarian dibagi dua:
 - Membagi rentang menjadi dua bagian di titik tengah (mid).
 - query(2 * node, ...) mencari ke **anak kiri** (rentang l hingga mid).
 - query(2 * node + 1, ...) mencari ke **anak kanan** (rentang mid + 1 hingga r).
- Menggabungkan hasil dari anak kiri dan anak kanan dengan fungsi max().

Ringkasan Strategi

Fungsi ini menggunakan pendekatan **Divide and Conquer** (Bagi dan Taklukkan) melalui rekursi:

1. Potong rentang pencarian secara berulang menjadi dua bagian.
2. Ambil nilai maksimum dari rentang yang relevan saja.
3. Buang rentang yang tidak relevan (dikembalikan sebagai -INF).

Studi Kasus F. Kandang Ayam Kandang Bebek

Deskripsi Cerita

Latar belakangnya menggunakan papan $2 \times N$, di mana ayam harus ke $(1,1) \rightarrow (1,0)$ dan bebek ke $(2,1) \rightarrow (2,0)$ tanpa ada petak yang dipotong/dilewati bersama (jalur ayam dan bebek tidak boleh saling bersilangan/sewarna).

Mari kita bedah dan jawab **Soal F1**, **Soal F2**, dan **Soal F3** satu per satu secara lengkap!

Soal F1

Pertanyaan: Manakah dari kondisi berikut yang **PASTI** mengakibatkan perselisihan (tidak ada rute tanpa tabrakan)?

- Papan berukuran 2×5 .
- Kandang Ayam di $(1,0)$ [masuk lewat $(1,1)$], Kandang Bebek di $(2,0)$ [masuk lewat $(2,1)$].

Analisis Kemungkinan Rute:

- **Pilihan A:** Ayam di $(1,5)$, Bebek di $(2,1)$.
 - Bebek di $(2,1)$ tinggal masuk ke $(2,0)$. Ayam berjalan di baris 1: $(1,5) \rightarrow (1,4) \rightarrow (1,3) \rightarrow (1,2) \rightarrow (1,1) \rightarrow (1,0)$. **(Bisa Tanpa Tabrakan)**
- **Pilihan B:** Ayam di $(1,2)$, Bebek di $(1,4)$.
 - Ayam lewat baris 1: $(1,2) \rightarrow (1,1) \rightarrow (1,0)$.
 - Bebek turun ke baris 2, lalu jalan ke kiri: $(1,4) \rightarrow (2,4) \rightarrow (2,3) \rightarrow (2,2) \rightarrow (2,1) \rightarrow (2,0)$. **(Bisa Tanpa Tabrakan)**
- **Pilihan C:** Ayam di $(2,5)$, Bebek di $(2,4)$.
 - Ayam naik ke baris 1 di kolom 5, lalu jalan di baris 1 ke kiri sampai $(1,1) \rightarrow (1,0)$.
 - Bebek jalan di baris 2 ke kiri sampai $(2,1) \rightarrow (2,0)$. **(Bisa Tanpa Tabrakan)**
- **Pilihan D:** Ayam di $(1,2)$, Bebek di $(2,4)$.
 - Ayam jalan di baris 1 ke $(1,1) \rightarrow (1,0)$.
 - Bebek jalan di baris 2 ke $(2,1) \rightarrow (2,0)$. **(Bisa Tanpa Tabrakan)**
- **Pilihan E:** Ayam di $(1,4)$, Bebek di $(1,2)$.
 - Ayam harus mencapai $(1,1)$. Karena posisinya di kolom 4, ia harus bergerak ke kiri melewati kolom 3, 2, dan 1.
 - Namun, Bebek berada di $(1,2)$! Untuk mencapai kandang bebek $(2,0)$, Bebek harus lewat $(2,1)$. Tetapi jika Bebek ingin turun ke baris 2, atau tetap di baris 1, Ayam di $(1,4)$ "terjebak" di sebelah kanan Bebek.
 - Ayam tidak bisa melompati Bebek atau melewati jalur di baris 2 tanpa memotong area jalan Bebek ke $(2,1)$. **Pasti Terjadi Perselisihan!**

Jawaban F1: E

Soal F2

Informasi Soal:

- Papan berukuran 2×1000 (total 2000 petak).
- Ada 1 Ayam di petak $(2,500)$.

- Ditanya: Berapa banyak posisi awal Bebek yang mungkin agar **TIDAK** terjadi perselisihan?

Analisis Rute:

1. Ayam berada di baris 2 kolom 500 (2,500).
2. Untuk mencapai pintu keluar Ayam di (1,1), Ayam **harus memotong ke baris 1** pada suatu kolom k (dengan $k \leq 500$), lalu berjalan di baris 1 ke arah kiri hingga (1,1).
3. Agar tidak menghalangi Ayam:
 - Bebek dapat memilih jalur di baris 2 sepenuhnya dari posisinya menuju (2,1).
 - Karena Ayam dari (2,500) akan *pindah* ke baris 1 di suatu titik $k \leq 500$, maka **seluruh petak di baris 2 dari kolom 1 hingga 500** dapat digunakan oleh Bebek (karena Bebek bergerak ke kiri di baris 2, sementara Ayam langsung berpindah ke baris 1 dan bergerak di baris 1 ke kiri).
 - Ayam cukup pindah ke baris 1 di kolom 500 (yaitu naik ke (1,500)) lalu menyusuri baris 1 dari kolom 500 sampai 1.
4. Dengan Ayam menyusuri seluruh **Baris 1 (kolom 1 ... 500)**, maka Ayam memblokir **seluruh Baris 1 dari kolom 1 sampai 500**.
5. Jadi:
 - Petak yang **aman dan bisa ditempati Bebek** adalah:
 - **Baris 2, kolom 1 ... 1000** (sebab Bebek di baris 2 manapun bisa terus berjalan di baris 2 sampai ke (2,1) tanpa pernah menyentuh baris 1 yang dikuasai Ayam). KECUALI petak (2,500) yang sudah diisi Ayam $\Rightarrow 1000 - 1 = 999$ petak.
 - **Baris 1, kolom 501 ... 1000** (sebab jika Bebek ada di baris 1 kolom > 500 , Bebek tinggal turun ke baris 2 di kolom tersebut, lalu jalan di baris 2 ke kiri sampai ke (2,1)) $\Rightarrow 500$ petak.

Perhitungan Total Posisi Bebek yang Valid:

Total Posisi = Posisi valid di Baris 2 + Posisi valid di Baris 1

$$\text{Total Posisi} = (1000 - 1) + (1000 - 500) = 999 + 500 = 1499$$

Jawaban F2: 1499

Soal F3

Informasi Soal & Matriks Kebecekan M (2×6):

$$\begin{pmatrix} 1 & 8 & 3 & 10 & 2 & 9 \\ 4 & 1 & 1 & 8 & 3 & 10 \end{pmatrix}$$

- **Ayam (1 ekor):** berada di (1,1).
- **Bebek (4 ekor):** berada di (1,5), (1,6), (2,5), dan (2,6).
- Ditanya: **Total nilai kerisihan minimal** untuk kelima hewan pulang tanpa perselisihan.

Analisis Rute Tanpa Perselisihan:

1. **Rute Ayam:**
 - Ayam berada di (1,1). Karena tujuan akhir ayam adalah (1,1) $\rightarrow$ (1,0), rute Ayam sangat singkat: hanya berada di petak (1,1).
 - Nilai kerisihan Ayam = $M_{1,1} = 1$.
2. **Rute 4 Bebek:**

- Semua Bebek harus menuju (2,1) tanpa menyentuh petak (1,1) (karena sudah dilewati Ayam).
- Semua Bebek bisa berjalan menuju (2,1) melalui **Baris 2!**
- Bebek di (1,5) harus turun ke (2,5) lalu jalan di baris 2 ke kiri: (2,5) → (2,4) → (2,3) → (2,2) → (2,1).
- Bebek di (1,6) harus turun ke (2,6) lalu jalan di baris 2 ke kiri: (2,6) → (2,5) → (2,4) → (2,3) → (2,2) → (2,1).
- Bebek di (2,5) jalan di baris 2 ke kiri: (2,5) → (2,4) → (2,3) → (2,2) → (2,1).
- Bebek di (2,6) jalan di baris 2 ke kiri: (2,6) → (2,5) → (2,4) → (2,3) → (2,2) → (2,1).

Perhitungan Rincian Kerisihan Bebek:

Mari kita hitung total nilai kebecakan untuk setiap rute Bebek:

- **Bebek 1 pada (2,5):**
 - Rute: (2,5) → (2,4) → (2,3) → (2,2) → (2,1)
 - Kerisihan = $M_{2,5} + M_{2,4} + M_{2,3} + M_{2,2} + M_{2,1} = 3 + 8 + 1 + 1 + 4 = 17$
- **Bebek 2 pada (2,6):**
 - Rute: (2,6) → (2,5) → (2,4) → (2,3) → (2,2) → (2,1)
 - Kerisihan = $M_{2,6} + \text{Kerisihan Bebek 1} = 10 + 17 = 27$
- **Bebek 3 pada (1,5):**
 - Rute: (1,5) → (2,5) → (2,4) → (2,3) → (2,2) → (2,1)
 - Kerisihan = $M_{1,5} + \text{Kerisihan Bebek 1} = 2 + 17 = 19$
- **Bebek 4 pada (1,6):**
 - Rute: (1,6) → (2,6) → (2,5) → (2,4) → (2,3) → (2,2) → (2,1) atau lewat (1,6) → (1,5) → (2,5) ...
 - Jalur 1 (turun ke 2,6): $M_{1,6} + \text{Kerisihan Bebek 2} = 9 + 27 = 36$
 - Jalur 2 (ke kiri ke 1,5 lalu turun): $M_{1,6} + M_{1,5} + \text{Kerisihan Bebek 1} = 9 + 2 + 17 = 28$ (*lebih murah*)

Total Kerisihan Seluruh Hewan:

$$\text{Total} = \text{Ayam} + \text{Bebek}(2,5) + \text{Bebek}(2,6) + \text{Bebek}(1,5) + \text{Bebek}(1,6)$$

$$\text{Total} = 1 + 17 + 27 + 19 + 28 = 92$$

Jawaban F3: 92

Soal Pemograman F:

Berikut adalah pembahasan dan solusi C++ lengkap untuk problem **Studi Kasus F: Kandang Ayam Kandang Bebek**.

Analisis Masalah

Sesuai dengan aturan pekarangan:

1. Ayam selalu berusaha menuju kandangnya di (1,0) (sehingga selalu melewati (1,1)).
2. Bebek selalu berusaha menuju kandangnya di (2,0) (sehingga selalu melewati (2,1)).
3. Seluruh pergerakan hewan hanya akan mengarah ke **Kiri, Atas, atau Bawah** karena semua nilai kebecakan lahan bernilai positif ($M_{i,j} \geq 1$), sehingga bergerak ke kanan hanya akan menambah biaya kerisihan.
4. Karena jalur ayam dan bebek tidak boleh bersilangan/saling timpa, maka "wilayah kekuasaan" rute ayam dan bebek dapat dibagi secara tegas menjadi dua bagian (atas untuk ayam, bawah untuk bebek).

Penentuan Titik Batas Wilayah (Split Point):

Misalkan lastA adalah kolom paling kanan yang memuat setidaknya 1 ayam, dan lastB adalah kolom paling kanan yang memuat setidaknya 1 bebek.

Kolom dari 1 hingga mid = min(lastA, lastB) **wajib terbagi dua secara horizontal**: Baris 1 secara mutlak hanya boleh dilewati oleh Ayam, dan Baris 2 secara mutlak hanya boleh dilewati oleh Bebek. Jika ada ayam di Baris 2 atau bebek di Baris 1 pada rentang wilayah kolom ini (1 s.d. mid), maka **perselisihan pasti terjadi** (keluarkan -1).

Untuk sisa kolom di sebelah kanan mid:

- Jika lastA > lastB, maka wilayah dari mid + 1 hingga lastA sepenuhnya merupakan hak milik Ayam. Ayam bebas bermanuver menggunakan Baris 1 maupun Baris 2 di rentang tersebut untuk mencari rute paling optimal (dengan *Dynamic Programming*).
- Berlaku sebaliknya jika lastB > lastA, rentang tersebut mutlak milik Bebek.

Kode Program C++

C++

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

using namespace std;

const long long INF = 1e18;

int main() {
    // Optimasi I/O
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int N;
    if (!(cin >> N)) return 0;

    string S1, S2;
    cin >> S1 >> S2;

    // Mengubah index string menjadi 1-based dengan padding
    S1 = " " + S1;
```

```

S2 = " " + S2;

vector<long long> M1(N + 1), M2(N + 1);
for (int i = 1; i <= N; ++i) cin >> M1[i];
for (int i = 1; i <= N; ++i) cin >> M2[i];

int lastA = 0, lastB = 0;
for (int i = 1; i <= N; ++i) {
    if (S1[i] == 'A' || S2[i] == 'A') lastA = max(lastA, i);
    if (S1[i] == 'B' || S2[i] == 'B') lastB = max(lastB, i);
}

int mid = min(lastA, lastB);

// Cek jika perselisihan PASTI terjadi di zona perpecahan (split region)
for (int i = 1; i <= mid; ++i) {
    if (S2[i] == 'A') { cout << -1 << "\n"; return 0; } // Ayam berada di wilayah mutlak Bebek
    if (S1[i] == 'B') { cout << -1 << "\n"; return 0; } // Bebek berada di wilayah mutlak Ayam
}

// Prefix sum untuk menghitung nilai kebecakan di zona split
vector<long long> PrefA(N + 1, 0), PrefB(N + 1, 0);
for (int i = 1; i <= N; ++i) {
    PrefA[i] = PrefA[i - 1] + M1[i];
    PrefB[i] = PrefB[i - 1] + M2[i];
}

// DP State: menyimpan biaya minimum dari cell(baris, kolom) menuju titik keluar di
(baris_hewan, mid)
vector<long long> distA1(N + 1, 0), distA2(N + 1, 0);
vector<long long> distB1(N + 1, 0), distB2(N + 1, 0);

// DP untuk jalur Ayam yang bermanuver leluasa di region kanan (jika ada)
if (lastA > mid) {
    distA1[mid] = M1[mid];
    distA2[mid] = INF; // Ayam tidak boleh masuk dari baris 2 di batas split
    for (int c = mid + 1; c <= lastA; ++c) {
        long long t1 = (distA1[c - 1] == INF) ? INF : M1[c] + distA1[c - 1];
        long long t2 = (distA2[c - 1] == INF) ? INF : M2[c] + distA2[c - 1];

        distA1[c] = t1;
        if (t2 != INF) distA1[c] = min(distA1[c], M1[c] + t2);

        distA2[c] = t2;
        if (t1 != INF) distA2[c] = min(distA2[c], M2[c] + t1);
    }
}

// DP untuk jalur Bebek yang bermanuver leluasa di region kanan (jika ada)
if (lastB > mid) {
    distB2[mid] = M2[mid];
    distB1[mid] = INF; // Bebek tidak boleh masuk dari baris 1 di batas split
    for (int c = mid + 1; c <= lastB; ++c) {
        long long t1 = (distB1[c - 1] == INF) ? INF : M1[c] + distB1[c - 1];
        long long t2 = (distB2[c - 1] == INF) ? INF : M2[c] + distB2[c - 1];

        distB1[c] = t1;
        if (t2 != INF) distB1[c] = min(distB1[c], M1[c] + t2);

        distB2[c] = t2;
        if (t1 != INF) distB2[c] = min(distB2[c], M2[c] + t1);
    }
}

```

```

    }
}

// Kalkulasi Total nilai kerisihan untuk setiap hewan individual
long long total_cost = 0;
for (int i = 1; i <= N; ++i) {
    if (S1[i] == 'A') {
        if (i <= mid) total_cost += PrefA[i];
        else total_cost += distA1[i] + PrefA[mid - 1];
    }
    if (S2[i] == 'A') { // (Hanya bisa terjadi di > mid karena di <= mid sudah difilter keluar -
1)
        total_cost += distA2[i] + PrefA[mid - 1];
    }
    if (S1[i] == 'B') { // (Hanya bisa terjadi di > mid karena di <= mid sudah difilter keluar -
1)
        total_cost += distB1[i] + PrefB[mid - 1];
    }
    if (S2[i] == 'B') {
        if (i <= mid) total_cost += PrefB[i];
        else total_cost += distB2[i] + PrefB[mid - 1];
    }
}

cout << total_cost << "\n";
return 0;
}

```

Kompleksitas

- **Waktu (Time Complexity):** $\mathcal{O}(N)$. Hanya ada sejumlah perulangan linear dari 1 sampai N yang digunakan untuk melakukan penentuan status lokasi, validasi awal, Prefix Sum, hingga iterasi *Dynamic Programming*. Batas eksekusi 3 detik sangat longgar untuk program $\mathcal{O}(N)$ sekecil ini yang umumnya diselesaikan di bawah ≈ 0.02 detik.
- **Ruang Memori (Space Complexity):** $\mathcal{O}(N)$ (mendekati 48 MB); Sangat aman untuk batas 256 MB. Menggunakan tipe data primitif *integer 64-bit* long long mengamankan risiko terjadinya nilai *Overflow* di saat tes parameter maksimum dimasukkan ($2 \cdot 10^5$ iterasi $\times$ 1,000,000 cost kebecakan).

INFORMASI TAMBAHAN TENTANG BAHASA PEMOGRAMAN C++

C++ :

```
ios_base::sync_with_stdio(false);
```

```
cin.tie(NULL);
```

Secara singkat: **Boleh dihilangkan, kode kamu tetap bisa berjalan dan memproses data dengan benar.**

Namun, **sangat TIDAK disarankan** untuk menghilangkannya jika kamu sedang mengerjakan **Soal Pemrograman / Competitive Programming** (seperti SONP, OSN, atau Codeforces).

Berikut adalah penjelasan lengkap mengapa 2 baris perintah tersebut ada dan apa dampaknya jika dihilangkan:

Apa Fungsi 2 Baris Perintah Tersebut?

Bawaannya, C++ dirancang agar bisa menggunakan fungsi Input/Output (I/O) gaya C++ (cin / cout) dan gaya C (scanf / printf) secara bersamaan dalam satu program tanpa mengacaukan urutan tampilan.

Agar urutannya selalu pas, C++ secara otomatis melakukan **sinkronisasi** (*synchronization*) dan **penambatan** (*binding*) di belakang layar. Proses sinkronisasi ini membutuhkan waktu eksekusi ekstra yang membuat cin dan cout menjadi cukup lambat.

1. `ios_base::sync_with_stdio(false);`
 - **Fungsi:** Mematikan sinkronisasi antara *stream* C++ (cin/cout) dan C (scanf/printf).
 - **Efek:** cin dan cout menjadi jauh lebih cepat (hampir menyamai kecepatan scanf/printf).
 - **Konsekuensi:** Kamu tidak boleh lagi mencampur penggunaan cin dengan scanf atau cout dengan printf dalam satu program.
2. `cin.tie(NULL);`
 - **Fungsi:** Melepas "ikatan" (*flush*) otomatis antara cin dan cout.
 - **Efek:** Secara default, setiap kali program menjalankan cin, C++ akan otomatis memikat cout untuk mencetak teks yang tertahan di *buffer* terlebih dahulu. Melepas ikatan ini membuat proses pembacaan input berulang-ulang menjadi jauh lebih cepat.

Kapan Boleh Dihilangkan?

Boleh dihilangkan tanpa masalah jika:

1. **Jumlah input/output sedikit:** Misalnya hanya membaca $N \leq 10.000$ baris data.
2. **Program biasa / Tugas Kuliah / Tugas Sekolah:** Di mana kecepatan waktu eksekusi dalam milidetik tidak dinilai secara ketat.
3. Kamu menggunakan fungsi I/O gaya bahasa C (scanf dan printf) sebagai gantinya.

Kapan JANGAN Dihilangkan?

Jangan dihilangkan jika kamu sedang mengerjakan soal kompetisi seperti **SONP / OSN** dengan kondisi:

1. **Jumlah data masukan sangat besar:** Misalnya $N = 200.000$ hingga 10^6 angka/karakter yang harus dibaca dari cin.

2. Jika dihilangkan, pembacaan cin yang lambat bisa menyebabkan program kamu terkena **TLE (Time Limit Exceeded)**, padahal logika algoritmanya sudah benar.

Rangkuman Perbandingan

Kondisi	Menggunakan 2 Baris Perintah	Tanpa 2 Baris Perintah
Kecepatan cin / cout	Sangat Cepat (Instan)	Lambat untuk data besar
Bisa pakai scanf & cin bersamaan?	TIDAK BISA (Bisa <i>bug</i> / urutan berantakan)	BISA
Risiko TLE pada data raksasa	Sangat Kecil	Sangat Tinggi

Saran Praktis: Saat mengerjakan soal-soal SONP/OSN, selalu biasakan untuk **tetap memasang 2 baris perintah** tersebut di bagian atas fungsi main(), dan gunakan murni cin serta cout (hindari endl, gunakan "\n" agar tetap cepat).

HEADER

Di dalam bahasa pemrograman C++, #include <algorithm> adalah sebuah *header file* bawaan (*standard library*) yang berisi kumpulan **fungsi-fungsi siap pakai untuk mengolah data secara cepat dan efisien**.

Sesuai namanya (*algorithm* = algoritma), library ini membebaskan kamu dari keharusan menulis logika algoritma dasar (seperti mengurutkan data, mencari nilai terbesar/terkecil, membalikkan urutan, dsb.) secara manual dari nol.

Mengapa Digunakan?

- Efisien & Cepat:** Algoritma di dalamnya dikembangkan dan dioptimasi oleh para ahli bahasa C++, sehingga kecepatannya jauh lebih unggul dibanding jika kita membuat fungsi sendiri.
- Kode Lebih Bersih & Singkat:** Menghemat puluhan baris kode (*boiler-plate code*).

Fungsi-Fungsi Populer di <algorithm>

Berikut adalah beberapa fungsi yang paling sering digunakan dalam pemrograman C++ sehari-hari maupun *Competitive Programming*:

1. Mengurutkan Data: std::sort()

Mengurutkan elemen (seperti array atau std::vector) secara menaik (*ascending*) atau menurun (*descending*).

- Kompleksitas:** $O(N \log N)$ (Sangat cepat!).

C++

```
#include <iostream>
#include <vector>
#include <algorithm> // Wajib di-include

using namespace std;

int main() {
    vector<int> angka = {5, 2, 8, 1, 3};
```

```
// Mengurutkan dari terkecil ke terbesar
sort(angka.begin(), angka.end()); // Hasil: 1, 2, 3, 5, 8

// Mengurutkan dari terbesar ke terkecil
sort(angka.rbegin(), angka.rend()); // Hasil: 8, 5, 3, 2, 1
}
```

2. Mencari Nilai Maksimum & Minimum: `std::max()` / `std::min()`

Mencari nilai terbesar atau terkecil antara dua bilangan atau dari kumpulan data.

C++

```
int a = 10, b = 25;
int terbesar = max(a, b); // Hasil: 25
int terkecil = min(a, b); // Hasil: 10

// Bisa juga untuk lebih dari dua angka:
int paling_besar = max({3, 15, 7, 99, 42}); // Hasil: 99
```

3. Mencari Elemen Terbesar/Terkecil di Array/Vector

- `std::max_element()` dan `std::min_element()`

C++

```
vector<int> v = {10, 50, 20, 80, 30};

// Mengembalikan iterator/pointer ke elemen terbesar
int maks = *max_element(v.begin(), v.end()); // Hasil: 80
int minn = *min_element(v.begin(), v.end()); // Hasil: 10
```

4. Membalikkan Urutan: `std::reverse()`

Membalikkan posisi elemen dari depan ke belakang.

C++

```
string s = "DENGKLEK";
reverse(s.begin(), s.end()); // s menjadi "KELKNED"
```

5. Pencarian Cepat pada Data Terurut: `std::binary_search()`

Mengecek apakah suatu nilai ada dalam data yang **sudah terurut**.

- **Kompleksitas:** $O(\log N)$.

C++

```
vector<int> v = {1, 3, 5, 7, 9}; // Wajib terurut!
```

```
bool ada = binary_search(v.begin(), v.end(), 5); // Mengembalikan true (1)
```

6. Mengisi Nilai Sama: `std::fill()`

Mengisi seluruh atau sebagian array/vector dengan nilai tertentu.

C++

```
vector<int> v(5);
fill(v.begin(), v.end(), 7); // Hasil: 7, 7, 7, 7, 7
```

Ringkasan

Fungsi	Kegunaan
sort()	Mengurutkan elemen
max() / min()	Membandingkan dua/lebih nilai
max_element() / min_element()	Mencari nilai ekstrim pada array/vector
reverse()	Membalikkan urutan
binary_search()	Mencari keberadaan elemen pada data terurut
lower_bound() / upper_bound()	Mencari batas/posisi elemen pada data terurut
next_permutation()	Membuat urutan permutasi berikutnya

Header <algorithm> merupakan salah satu library yang **paling sering digunakan bersama** <vector> dalam penyelesaian soal-soal kompetisi sains (seperti OSN/SONP).

BILANGAN TIDAK TERHINGGA

Di dalam pemrograman C++ (khususnya pada *Competitive Programming* atau saat membuat algoritma), perintah `const int INF = 1e9`; digunakan untuk membuat sebuah **nilai konstan yang mewakili nilai Tak Hingga** (∞). (lihat soal Pemograman E)

Berikut adalah rincian penjelasannya:

1. Membedah Komponen Perintah

- `const`: Menyatakan bahwa variabel ini bersifat **konstan** (nilainya tetap dan tidak bisa diubah-ubah di tengah program).
- `int`: Tipe data bilangan bulat (*integer*).
- `INF`: Nama variabel (singkatan umum dari **Infinity** / Tak Hingga).
- `1e9`: Bentuk notasi eksponensial matematis dari 1×10^9 atau 1.000.000.000 (**1 Miliar**).

2. Mengapa Digunakan dalam Algoritma?

Komputer tidak memiliki simbol nilai "Tak Hingga" (∞) secara alami untuk bilangan bulat. Oleh karena itu, para pemrogram membuat sebuah **angka yang sangat besar** sebagai representasi *infinity*.

Nilai INF ini paling sering digunakan dalam 2 kondisi utama:

A. Untuk Inisialisasi Pencarian Nilai Minimum

Saat kita ingin mencari nilai terkecil (minimum) dari suatu kumpulan data atau dalam *Dynamic Programming* (seperti pada soal **TebasTebang Bambu** sebelumnya), kita menset variabel penampung awal dengan nilai INF.

Dengan begitu, angka pertama apa pun yang kita periksa nanti **pasti lebih kecil** dari INF dan akan menggantikan nilai awal tersebut.

C++


```
int min_val = INF; // Diisi 1 Miliar dulu

for (int x : data) {
    if (x < min_val) {
        min_val = x; // Angka pertama pasti akan mengalahkan 1 Miliar
    }
}
```

B. Menandai Kondisi "Tidak Mungkin" atau "Belum Terjangkau"

Dalam algoritma graf (seperti *Dijkstra* atau *Floyd-Warshall*) dan *Dynamic Programming*, nilai INF digunakan sebagai penanda bahwa sebuah kota/node **belum bisa dijangkau** atau suatu keadaan **tidak mungkin terjadi**.

(Pada kode soal bambu sebelumnya, kita menggunakan -INF / -1e9 sebagai penanda bahwa jumlah ikat bambu awal di posisi tersebut **tidak mungkin dibentuk**).

3. Mengapa Memilih 10^9 (1e9), Bukan Batas Maksimal INT_MAX (2×10^9)?

Di C++, batas maksimum tipe data int sebenarnya adalah INT_MAX ($\approx 2,14 \times 10^9$). Namun, pemrogram lebih suka menggunakan 1e9 (10^9) karena alasan **keamanan dari Overflow**:

- 1. **Aman Jika Ditambah:** Jika kamu melakukan operasi penambahan seperti $INF + 5$ atau $INF + cost$, nilainya menjadi 1.000.000.005. Angka ini **masih muat** di dalam int.
- 2. **Bahaya INT_MAX:** Jika kamu menggunakan INT_MAX lalu menambahkannya dengan angka 1 saja ($INT_MAX + 1$), variabel akan mengalami **Integer Overflow** dan nilainya akan memutar menjadi bilangan negatif (sekitar $-2,14 \times 10^9$), yang menyebabkan *bug* fatal pada program!

Rangkuman

Karakteristik	Penjelasan
Arti	Mewakili nilai Tak Hingga (∞)
Nilai Asli	1.000.000.000 (1 Miliar)
Kegunaan Utama	Inisialisasi pencarian nilai minimum / penanda kondisi tidak valid
Alasan Angka 10^9	Cukup besar untuk mayoritas soal, namun aman dari <i>integer overflow</i> saat dijumlahkan